Table of Contents

4.1 Plan Introduction **1**
4.1.1 Project Deliverables 1
4.2 Project Resources **2**
4.2.1 Hardware Resources 3
4.2.2 Software Resources 3
4.3 Project Organization **4**
4.4 Project Schedule **5**
4.4.1 PERT / GANT Chart 5
4.4.2 Task/ Resource Table 7

4.1 Plan Introduction

This Software Development Plan (SDP) outlines the development process, resources, team responsibilities, and schedule for MD-Gooey, a web-based molecular dynamics simulation management platform designed for Dr. Alavi's research team at Loyola Marymount University.

The purpose of MD-Gooey is to simplify the process of preparing, submitting, executing, and analyzing molecular dynamics simulations using GROMACS. Currently, researchers must navigate multiple devices and platforms to configure and execute simulations. MD-Gooey aims to streamline this workflow by providing a centralized web application where users can create simulation requests, monitor their progress, and retrieve results. The project consists of three primary components: MD-Gooey, the user-facing web application responsible for queuing and executing simulations.

Development will follow an Agile methodology, with the primary goal of completing and integrating the system's core functionality by the end of the Fall 2026 semester. Major development activities will include implementing the database and API, expanding the existing web application, developing the simulation execution queue, and integrating all three components. Functional and performance testing will take place throughout development, with feedback from Dr. Alavi and the course instructor guiding revisions and improvements. Additional refinements may continue after the initial implementation.

4.1.1 Project Deliverables

The following deliverables will be completed throughout the Fall 2026 semester. These deliverables include the development of 3 primary software components, their integration into a functioning system, and the documentation necessary to describe and demonstrate the project.

1. First Iteration — Application Structure and Database Foundation (10/23)
    - Implement React routing to support navigation between pages within the MD-Gooey web application.
    - Deploy the updated frontend application.

    - Implement functionality to appropriately handle partially created simulations.
    - Develop a self-contained database and API layer using Neon PostgreSQL and Express.js.
2. Second Iteration — Authentication and Initial Integration (Version 2 Complete) (10/30)
    - Implement user authentication within the database and frontend application.
    - Connect the MD-Gooey frontend to MD-GooPI to support communication with the database.
    - Develop the first iteration of the MD-Queuey background daemon responsible for retrieving and processing simulation requests.
    - Test the connection between the frontend and database to verify successful communication and data handling.
    - Coordinate with LMU Information Technology Services (ITS) to run the initial daemon implementation on the workstation.
3. Third Iteration — Daemon Refinement and Database Integration (11/8)
    - Identify and resolve issues discovered during the initial daemon execution.
    - Refine MD-Queuey to ensure proper integration with the database and API layer.
    - Coordinate with ITS to rerun the updated daemon and evaluate its functionality.
4. Fourth Iteration — Final Integration and Deployment (Version 3 Complete) (11/20)
    - Resolve remaining issues identified during previous testing and integration.
    - Coordinate with ITS to perform a final test of the daemon on the LMU workstation.
    - Deploy the complete system architecture, integrating MD-Gooey, MD-GooPI, and MD-Queuey.
    - Verify that the primary components function together as intended.
5. Project Documentation and Demonstration (12/10)
    - Complete the required software engineering documentation, including the Software Requirements Specification (SRS) and Software Development Plan (SDP).
    - Document the final system implementation and testing results as required by the course.
    - Demonstrate the integrated MD-Gooey system and its core functionality to the instructor and project stakeholders.

4.2 Project Resources

The development and deployment of MD-Gooey will require a combination of hardware and software resources to support the implementation, integration, and testing of the system's three primary components: MD-Gooey, MD-GooPI, and MD-Queuey. Development team members will use personal computers and software development tools to implement and test the application, while cloud-hosted services will support the database, API, and frontend deployment. The project will also utilize a dedicated multi-GPU workstation at Loyola Marymount University to execute molecular dynamics simulation through GROMACS. The following sections describe the hardware and software resources required throughout the development process.

4.2.1 Hardware Resources

The development of MD-Gooey will utilize personal computers owned by the development team, along with a dedicated workstation provided by Loyola Marymount University. Team members will use their personal laptops and desktops for software development, debugging, and preliminary testing. The LMU workstation will be used to execute molecular dynamics simulations through GROMACS and verify the integration and functionality of MD-Queuey with the database and API.

4.2.1.1 Personal Development Computers

Each team member will use their own laptop or desktop computer to develop and test the MD-Gooey frontend, MD-GooPI service layer, and MD-Queuey application.These computers will provide necessary environments for writing code, running local development servers, managing the project repository, and testing individual components before integration with the LMU workstation

Specific hardware configurations will vary depending on the development machine used by each team member.

4.2.1.2 LMU Simulation Workstation

The project will utilize a dedicated LMU workstation for executing GROMACS simulations and testing the integration of the complete system. The workstation has the following hardware specifications.

Component	Specification
Processor	Two Intel Xeon Gold 6538Y + processors
CPU Cores	64 physical cores, 128 logical threads
Memory	755 Gib RAM
Graphics Processing Units	Four NVIDIA RTX 5000 Ada Generation GPUs
GPU Memory	32 GB VRAM per GPU
Storage	3.5 TB filesystem, with approximately 2.6 TB available at the time of inspection

The workstation will serve as the execution environment for MD-Queuey and GROMACS. Testing on this machine will verify that simulation requests can be retrieved, queued, executed, and processed successfully. Access to the workstation for deployment and testing will be coordinated through LMU Information Technology Services(ITS).

4.2.2 Software Resources

The development of MD-Goeey will require various software tools, programming languages, frameworks, and cloud-based services to support the development, testing, and deployment of the system. These resources will be used across three primary components: MD-Gooey, MD-GooPI, and MD-Queuey.

4.2.2.1 Development Tools and Programming Languages

- **Visual Studio Code (VS Code):** The primary integrated development environment (IDE) used for writing, debugging, and maintaining project code.
- **JavaScript and TypeScript:** Programming languages used in the development of the web application and API.
- **Python:** Used to develop the MD-Queuey background daemon responsible for retrieving and processing simulation requests.
- **Git and Github:** Used for version control, collaborative development, source code management, and tracking changes throughout the project.

4.2.2.2 Frontend Development

- **React:** A JavaScript library used to develop and extend the MD-Gooey user interface.
- **React Router:** Used to manage navigation between pages, including authentication, simulation creation, and the My Simulations dashboard.

4.2.2.3 Backend and Database

- **Node.js**: The JavaScript runtime environment used to execute the backend API.
- **Express.js:** The framework used to develop MD-GooPI's API endpoints for handling simulation requests and database interactions.
- **Neon PostgreSQL:** A cloud-hosted relational database used to store user information, simulation requests, execution statuses, and simulation results.
- **Vercel:** The cloud hosting platform used to deploy the API and support communication between the web application and database.

4.2.2.4 Simulation Execution Environment

- **Ubuntu 22.04.5 LTS:** The operating system installed on the LMU workstation.
- **GROMACS 2025.3:** The molecular dynamics simulation software used to execute submitted simulation requests.
- **CUDA 13.0:** NVIDIA's GPU computing platform used to support GPU-accelerated simulations.
- **Open MPI 5.0.8:** The message-passing implementation used to support parallel execution of molecular dynamics simulations.
- **OpenMP:** Used by GROMACS for multithreaded CPU processing.
- **NVIDIA Driver 580.126.09:** The workstation's GPU driver, supporting execution on its four NVIDIA RTX 5000 Ada Generation GPUs.

These software resources will support implementing and integrating MD-Gooey's frontend, database, API, and simulation execution components throughout the Fall 2026 semester.

4.3 Project Organization

The development of MD-Gooey will be carried out collaboratively by the project team, with responsibilities distributed across the three primary software components: MD-Gooey, MD-GooPI, and MD-Queuey. Team members will be assigned development tasks based on their technical experience and project requirements. While sub-teams will be responsible for specific components, the team will collaborate on system integration, testing, debugging, and documentation to ensure that all components function together:

MD-Gooey Frontend----React Routing, GH Pages Deployment, UI, Auth integration, Sim management -> **Jackson & Laith & Jake**

MD-GooPI Backend---Neon database configuration, Express.js. API implementation -> **Marcus & Eldrick**

MD-Queuey Daimon---Unpacking simulation data, creating executable bash script to run simulation with GROMACS, appropriate sandboxing, monitoring simulation status, sending status back to the database -> **Jake & Eldrick**

System integration---Connecting the frontend,databaseAPI, and Daimon-> **Jackson & Marcus & Eldrick**

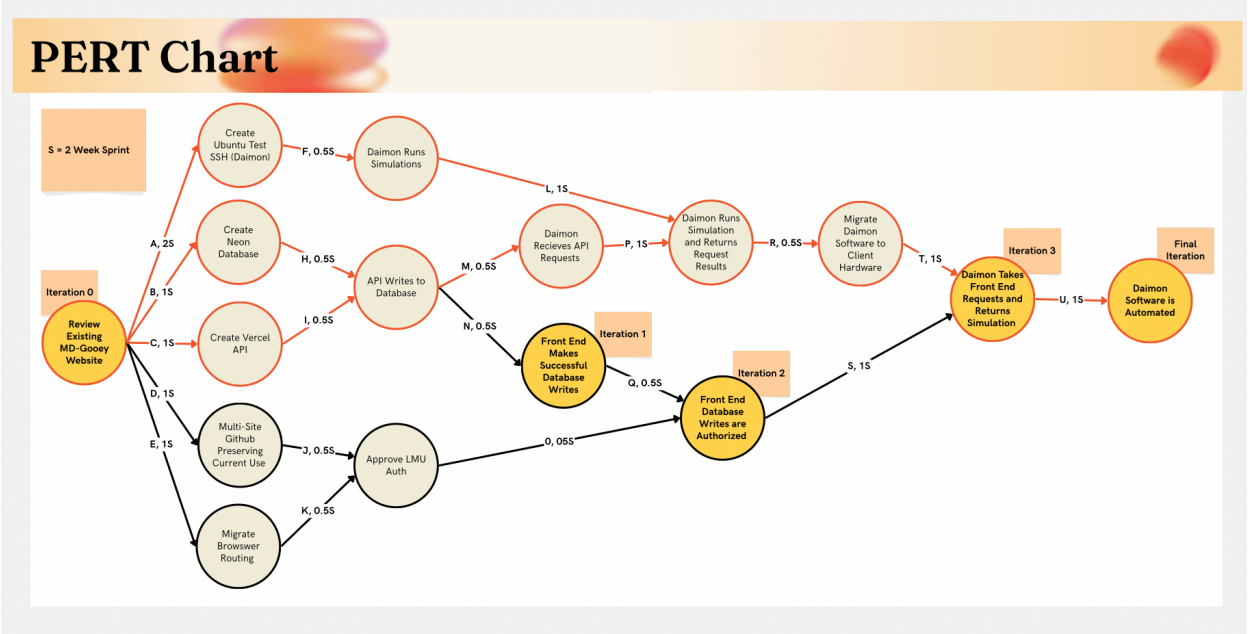
Testing and Debugging ---Testing system component, resolving issues -> **Whole team**

Documentation Maintain the SRS, SDP, technical documentation -> **Whole team**

4.4 Project Schedule

This section details the scheduling and development plan for MD-Gooey.

4.4.1 PERT / GANT Chart



Sprints = 2 Weeks				
Task	Optimistic Time	Most Likely Time	Pessimistic Time	Expected Time
Create Ubuntu Test SSH (Daimon)	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Create Neon Database	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Create Vercel API	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Multi-Site Github	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Migrate Browser Routing	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Daimon Runs SImulation	0.25 Sprint	0.5 Sprint	1 Sprint	0.5 Sprint
API Writes to Database	0.25 Sprint	0.5 Sprint	1 Sprint	0.5 Sprint
Approve LMU Auth	0.25 Sprint	0.5 Sprint	1 Sprint	0.5 Sprint
Daimon Receives API Requests	0.25 Sprint	0.5 Sprint	1 Sprints	0.5 Sprint
Front End Makes Database Writes	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Daimon Runs and Returns Simulation Results	0.25 Sprint	0.5 Sprint	1 Sprints	0.5 Sprint
Front End Database Writes are Authorized	0.25 Sprint	0.5 Sprint	1 Sprint	0.5 Sprint
Migrate Daimon to Client Hardware	0.25 Sprint	0.5 Sprint	1 Sprint	0.5 Sprint
Daimon Takes Front End Requests and Returns Simulations	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Daimon Software is Automated	0.5 Sprint	1 Sprint	2 Sprints	1 Sprint
Critical Path Total	3 Sprints	7 Sprints	14 Sprints	7 Sprints

4.4.2 Task / Resource Table

Task	Software Resource	Hardware Resource	Assignee(s)
Create Ubuntu Test SSH (Daimon)	Ubuntu 22.04.5 LTS	Personal Computer	Marcus & Eldrick
Create Neon Database	Neon PostgreSQL, Node.js, Express.js	Personal Computer	Marcus & Eldrick
Create Vercel API	Vercel	Personal Computer	Marcus & Eldrick
Multi-Site Github	Github	Personal Computer	Jackson & Laith & Jake
Migrate Browser Routing	React, React Router	Personal Computer	Jackson & Laith & Jake
Daimon Runs SImulation	Ubuntu 22.04.5 LTS, GROMACS 2025.3, Open MPI 5.0.8	Personal Computer	Jake & Eldrick
API Writes to Database	Neon PostgreSQL, Node.js, Express.js, Vercel	Personal Computer	Jackson & Laith & Jake
Approve LMU Auth	Github, React, React Router	Personal Computer	Jackson & Laith & Jake
Daimon Receives API Requests	Ubuntu 22.04.5 LTS, Neon PostgreSQL, Node.js, Express.js	Personal Computer	Jake & Eldrick
Front End Makes Database Writes	React, Neon PostgreSQL, Node.js, Express.js	Personal Computer	Jackson & Marcus
Daimon Runs and Returns Simulation Results	Neon PostgreSQL, Node.js, Express.js	Personal Computer	Jake & Eldrick
Front End Database Writes are Authorized	React, Neon PostgreSQL, Node.js, Express.js	Personal Computer	Jackson & Marcus

Migrate Daimon to Client Hardware	Ubuntu 22.04.5 LTS	Client Ubuntu SSH	Jake & Eldrick
Daimon Takes Front End Requests and Returns Simulations	Ubuntu 22.04.5 LTS, Neon PostgreSQL, Node.js, Express.js	Client Ubuntu SSH	Whole Team
Daimon Software is Automated	Ubuntu 22.04.5 LTS, Neon PostgreSQL, Node.js, Express.js	Client Ubuntu SSH	Whole Team